



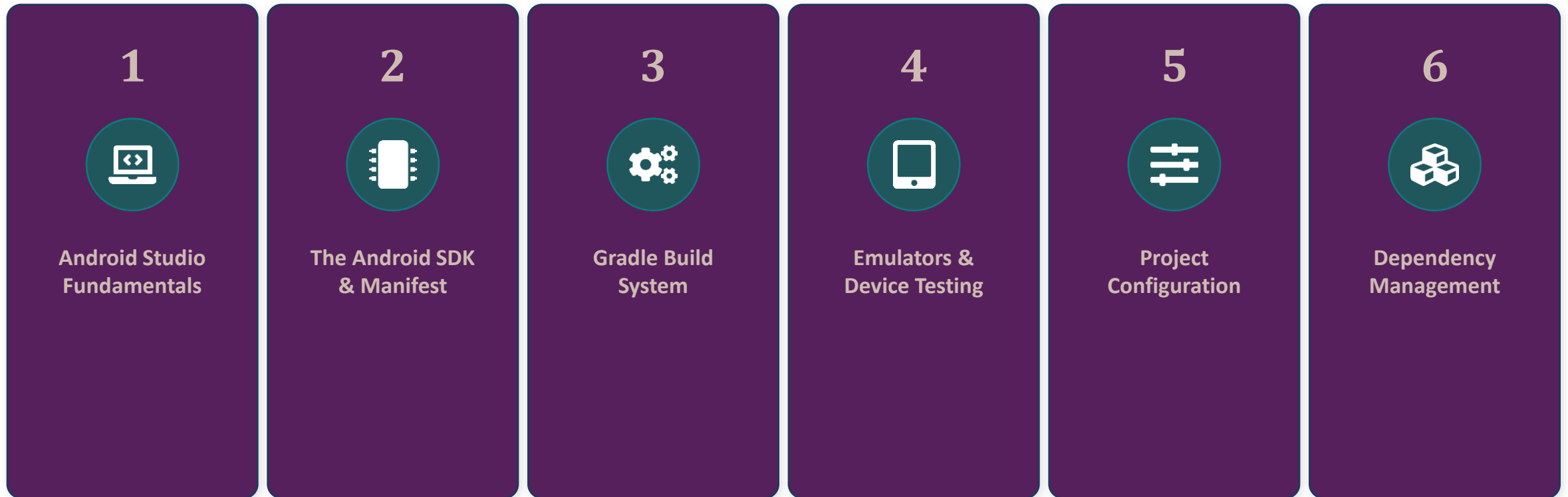
MODULE 11

Android Development Environment

Phase 03 | Android Development with Kotlin

Module 11 Roadmap

Six blocks. By the end: a fully configured, multi-environment PNC Mobile Android project.



Android Studio Fundamentals

- The IDE Tour
- Project Structure
- Logcat
- Essential Shortcuts



Android Studio: Built on IntelliJ



Project panel

Left side. File tree, but organized by Android-specific groupings (manifests, java/kotlin, res) rather than raw folders by default.



Editor

Center. Same Kotlin editing experience as IntelliJ, plus XML layout editors and a visual Compose preview.



Logcat

Bottom. Android's real-time device/emulator log stream - the direct equivalent of Xcode's console, and used constantly.



Build panel

Bottom. Shows Gradle sync and build output - where most early troubleshooting actually happens.



Logcat: Reading the Stream

- TIP: Filter Logcat by package name (com.pnc.mobileapp) FIRST, before adding a tag filter
 - Ctrl-Space in the filter box brings up suggestions
 - package:mine filters for the current project ids
 - The system alone produces hundreds of lines per second that have nothing to do with your app

Code Bite

```
// A tagged log line,  
// filterable by tag in Logcat's search bar  
Log.d("AccountsRepository",  
      "Fetched ${accounts.size} accounts")  
Log.e("AccountsRepository",  
      "Fetch failed", exception)  
  
// Log levels, least to most severe:  
// Verbose -> Debug -> Info -> Warn  
//          -> Error -> Assert
```



Block 1 - Android Studio Fundamentals

Skill Check ✓

- What IDE is Android Studio built on top of?
- What is Logcat the direct equivalent of, from Module 5's iOS tooling?
- Why filter Logcat by package name before adding any other filter?

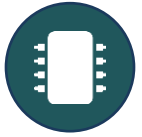


The Android SDK & App Manifest

- API Levels & Version Compatibility
- AndroidManifest.xml
- Permissions



API Levels & Version Compatibility



minSdk

The oldest Android version the app will install and run on at all



targetSdk

The API level the app is built and tested against - opts into that version's behavior changes



compileSdk

The SDK version used to compile the code - usually the latest stable release

A realistic enterprise floor, mirroring Module 5's iOS 16 discussion: minSdk 26 (Android 8.0) covers the large majority of active devices while allowing modern Kotlin/Compose APIs.



AndroidManifest.xml: Not Optional Boilerplate

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.CAMERA" />
    <uses-permission android:name="android.permission.INTERNET" />
    <application
        android:label="PNC Mobile"
        android:icon="@mipmap/ic_launcher">
        <activity android:name=".MainActivity" android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```



Specifying Device Features Used

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">  
    <uses-feature  
        android:name="android.hardware.camera"  
        android:required="true" />  
  
    <uses-feature  
        android:name="android.hardware.bluetooth"  
        android:required="false" />  
  
    <uses-feature  
        android:name="android.hardware.fingerprint"  
        android:required="false" />  
  
    <uses-feature  
        android:name="android.hardware.sensor.accelerometer"  
        android:required="false" />  
  
</manifest>
```



Specifying Permissions Needed

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">  
  
    <uses-permission android:name="android.permission.CAMERA" />  
    <uses-permission android:name="android.permission.INTERNET" />  
    <uses-permission android:name="android.permission.BATTERY_STATS" />  
    <uses-permission android:name="android.permission.READ_CONTACTS" />  
  
</manifest>
```



Block 2 - The Android SDK & Manifest

Skill Check ✓

- What's the difference between minSdk and targetSdk?
- What happens if an app calls the camera without declaring the CAMERA permission in the manifest - how is this different from Module 5's iOS crash behavior?



Gradle Build System

- build.gradle.kts Fundamentals
- Build Types
- Product Flavors
- Reading a Sync Error



build.gradle.kts: the Project's Recipe

```
plugins {  
    id("com.android.application")  
    id("org.jetbrains.kotlin.android")  
}  
  
android {  
    namespace = "com.pnc.mobileapp"  
    compileSdk = 34  
    defaultConfig {  
        applicationId = "com.pnc.mobileapp"  
        minSdk = 26  
        targetSdk = 34  
        versionCode = 1  
        versionName = "1.0"  
    }  
}
```

```
dependencies {  
    implementation("androidx.core:core-ktx:1.13.1")  
}
```



Build Types & Product Flavors



Build Types

debug (fast, debuggable) vs. release (optimized, minified, signed for distribution)



Product Flavors

dev / staging / prod - different applicationId suffix, API base URL, app name per flavor



Build Variant

The combination: devDebug, stagingDebug, prodRelease, etc. - each independently selectable and runnable



Adding Build Types to Gradle

```
android {  
    buildTypes {  
        release {  
            minifyEnabled false  
        }  
        debug {  
            applicationIdSuffix = '.debug'  
        }  
        staging {  
            initWith release  
            applicationIdSuffix = '.staging'  
        }  
    }  
}
```



Adding Product Flavors to Gradle

```
flavorDimensions "tier", "environment"
productFlavors {
    create("dev") {
        dimension = "environment"
        applicationIdSuffix = ".dev"
    }
    create("prod") {
        dimension = "environment"
    }
    create("free") {
        dimension = "tier"
        applicationIdSuffix = ".free"
    }
    create("paid") {
        dimension = "tier"
        applicationIdSuffix = ".paid"
    }
}
```



Reading a Sync Error

- 1 Read the FIRST line of the error, not the full stack - it usually names the actual problem
- 2 Check for a red squiggle directly in the build.gradle.kts file itself before searching online
- 3 Try File > Sync Project with Gradle Files again - a surprising fraction of errors are transient
- 4 If a dependency version is the issue, check it actually exists at that version on the declared repository



Block 3 - Gradle Build System

Skill Check ✓

- What's the difference between what a build type controls and what a product flavor controls?
- What is 'prodRelease' an example of?
- What's the first thing to check when a Gradle sync fails?



Emulators & Device Testing

- The AVD Manager
- Emulator Capabilities & Limits
- Physical Device Testing & USB Debugging



The AVD Manager

- 1 Tools > Device Manager → Create Virtual Device
- 2 Choose a device definition (screen size/density) - e.g. Pixel 8, or a smaller/older device for compatibility testing
- 3 Choose a system image - this is the specific Android API level/version to emulate
- 4 Name it and finish - the AVD now appears in the device dropdown, ready to launch



What the Emulator Can and Cannot Do

Good For

- Rapid iteration during UI development
- Testing across many screen sizes, densities, and API levels
- Automated instrumented tests in CI pipelines

Requires a Physical Device

- True camera and sensor behavior
- Push notification delivery at real-world latency
- Accurate battery and thermal performance profiling
- Real biometric hardware (fingerprint, face unlock)
- True cellular network conditions and carrier variability



Physical Device Testing & USB Debugging



Enable Developer Options

Settings > About Phone > tap Build Number 7 times



Enable USB Debugging

Settings > Developer Options > USB Debugging



Authorize the computer

A dialog appears on the device on first connection - must be accepted



Block 4 - Emulators & Device Testing

Skill Check ✓

- Name two things you can only truly validate on a physical Android device, not the emulator.
- What two pieces of information does an AVD's system image specify?
- What Android-specific ritual enables Developer Options on a physical device?



Project Configuration

- Environment-Specific Configuration
- BuildConfig Fields
- Signing & App Bundles



Creating Environment-Specific Config Fields

```
productFlavors {  
    create("dev") {  
        dimension = "environment"  
        applicationIdSuffix = ".dev"  
        buildConfigField("String", "BASE_URL", "\"https://api-dev.pncmobile.com\"")  
    }  
    . . .  
}
```

```
buildFeatures {  
    compose = true  
    buildConfig = true  
}
```



Using Environment Configuration Fields

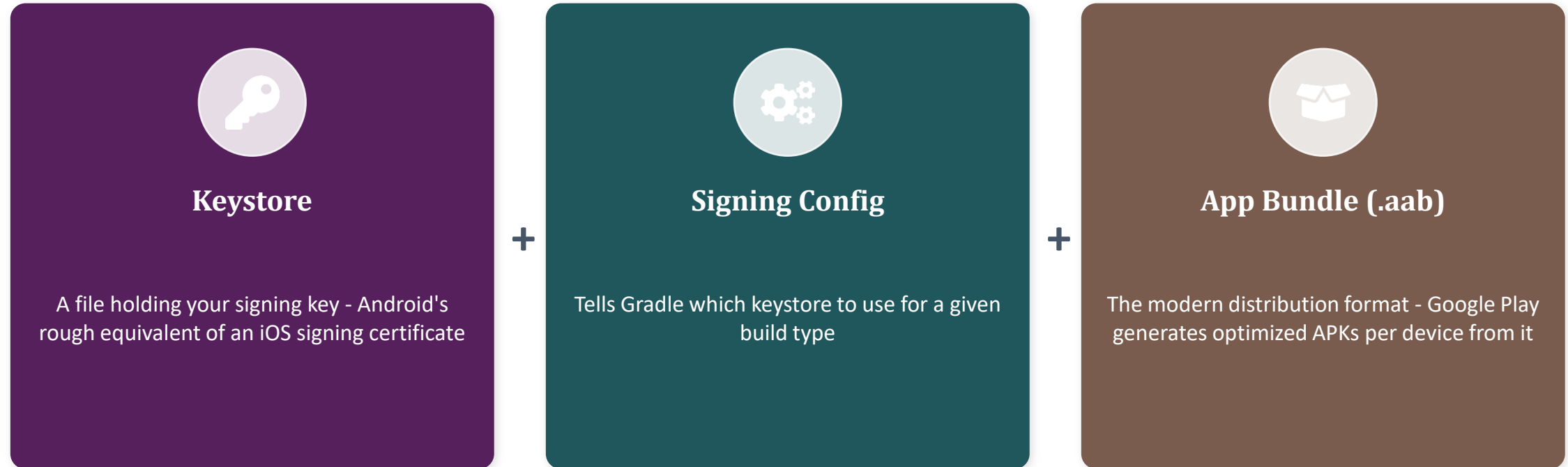
- Once the config fields have been defined you must:
 - Sync the Project with Gradle files
 - Clean and rebuild the project
- Then the global BuildConfig object will have your custom field(s) added as properties

Code Bite

```
// Usage anywhere in the app:  
val baseUrl = BuildConfig.BASE_URL
```



Signing & App Bundles



Block 5 - Project Configuration

Skill Check ✓

- How would you switch PNC Mobile Android from talking to the dev API to the staging API, with zero code changes?
- What does applicationIdSuffix accomplish, and why does it matter for installing dev/staging/prod builds side by side?
- What's the modern format Google Play expects apps to be distributed in?



Dependency Management

- Gradle Dependencies & Maven Central
- Version Catalogs
- Evaluating a Third-Party Library



Gradle Dependencies & Maven Central

```
dependencies {  
    implementation("com.squareup.retrofit2:retrofit:2.11.0")  
    implementation("androidx.compose.ui:ui:1.7.0")  
}
```

```
// implementation: this dependency is NOT exposed to modules that  
// depend on this one - the default, and usually correct, choice.  
//  
// api: this dependency IS exposed transitively - use only when a  
// downstream module genuinely needs to see it directly.
```



Version Catalogs: Centralizing Dependency Versions

```
# libs.versions.toml
```

```
[versions]
```

```
retrofit = "2.11.0"
```

```
[libraries]
```

```
retrofit = { module = "com.squareup.retrofit2:retrofit", version.ref = "retrofit" }
```

```
# In build.gradle.kts:
```

```
dependencies {
```

```
    implementation(libs.retrofit)
```

```
}
```



Evaluating a Third-Party Library



License compatibility

MIT/Apache 2.0 generally safe; GPL-family often needs legal sign-off



Maintenance activity

Recent commits? Are critical issues being answered?



APK size & method count impact

Every dependency affects app size and, historically, the 64K method limit



Data handling & privacy

What does it collect, and where does it send it?



Transitive dependencies

You inherit its entire dependency tree's risk



Version pinning

Pin an exact or ranged version - never an unbounded "latest" in production



Block 6 - Dependency Management

Skill Check ✓

- What's the practical difference between implementation and api when declaring a dependency?
- What problem does a Gradle version catalog (libs.versions.toml) solve?
- A dependency has an unclear license and hasn't been updated in three years. What's the correct next step?

